AI-BASED SELF-HEALING IT INFRASTRUCTURE FRAMEWORK

PHASE I REPORT

Submitted by

MANASA VISAKAI S 230701172

MAHALAKSHMI V 230701171

MARIA PRISCILLA M 230701180

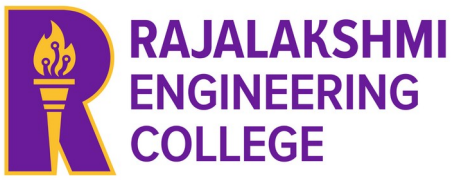
MAGESH VASAN M 230701507

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**AI-BASED SELF-HEALING IT INFRASTRUCTURE FRAMEWORK**" is the bonafide work of **MANASA VISAKAI S (230701172), MAHALAKSHMI V (230701171), MARIA PRISCILLA M (230701180) and MAGESH VASAN M (230701507),** who carried out the work under my supervision. Certified further that, to the best of our knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini,
Professor &
Head of The Department,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602105.

SIGNATURE

Ms. Aswana Lal,
Supervisor
Assistant Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602105.

Submitted to Project Viva Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Modern IT services depend on a chain of interconnected components such as web servers, application programming interfaces, databases, background workers and the containers that host them. When one of these components becomes overloaded, unavailable or behaves abnormally, conventional operations rely on an administrator to notice an alert, inspect metrics and logs, identify the cause and perform a recovery step manually, which increases the Mean Time To Recover (MTTR). This report presents an AI-Based Self-Healing IT Infrastructure Framework that connects monitoring, machine-learning analysis and controlled recovery in one automated workflow. A representative microservice environment consisting of an Nginx web server, a Python FastAPI service, a PostgreSQL 15 database and a Python healing worker is deployed with Docker Compose. Prometheus collects host, container and application telemetry through Node Exporter, cAdvisor and a FastAPI instrumentator. A data-processing module converts the telemetry into nine features that are analysed by an Isolation Forest model for anomaly detection, an XGBoost classifier for failure-risk prediction and a rule-based Decision Tree for root-cause analysis. When an incident is confirmed, the self-healing engine executes an allow-listed container restart, re-checks service health and records the incident, root cause, action, status and recovery duration in a PostgreSQL audit trail that is displayed on a live dashboard. In the recorded test run, all six logged incidents affecting the API and web services were automatically recovered, with a mean recovery duration of 4.58 seconds. By combining continuous telemetry, layered AI decisions and verified recovery, the framework reduces dependence on manual first-level troubleshooting while keeping every automated action observable and auditable.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Ms. Aswana Lal**, Assistant Professor, Department of Computer Science and Engineering for her valuable guidance, constant support and useful suggestions throughout the course of the project and during our reviews.

MANASA VISAKAI S : 230701172

MAHALAKSHMI V : 230701171

MARIA PRISCILLA M : 230701180

MAGESH VASAN M : 230701507

TABLE OF CONTENTS

CHAPTER NO.		TITLE	PAGE NO.
		ABSTRACT	**ii**
		LIST OF TABLES	**vi**
		LIST OF FIGURES	**vii**
		LIST OF ABBREVIATIONS	**viii**
1		**INTRODUCTION**	**1**
	1.1	General	1
	1.2	Objectives	1
	1.3	Existing System	2
	1.4	Proposed System	3
	1.5	Scope of the Project	3
	1.6	Organization of the Report	3
	1.7	Problem Statement	4
2		**LITERATURE SURVEY**	**5**
	2.1	Literature Review	5
	2.2	Comparison and Discussion	7
	2.3	Conclusion	8
3		**PROPOSED SYSTEM**	**9**
	3.1	System Architecture	9
	3.2	System Requirements	10
	3.2.1	Software Requirements	10
	3.2.2	Hardware Requirements	10
	3.2.3	Dataset and Feature Requirements	10
	3.3	System Design	11
	3.3.1	Activity Diagram	11
	3.3.2	Data Flow Diagram	12
	3.3.3	Incident Record Design	13
4		**PROJECT DESCRIPTION**	**14**
	4.1	Methodology	14
	4.2	Main Modules of the Project	14
	4.2.1	Infrastructure Simulation	14
	4.2.2	Monitoring and Data Collection	15
	4.2.3	Data Processing	15
	4.2.4	AI Anomaly Detection	15

CHAPTER NO.		TITLE	PAGE NO.
	4.2.5	Failure Prediction	15
	4.2.6	Root Cause Analysis	15
	4.2.7	Self-Healing and Recovery	15
	4.2.8	Dashboard and Incident Management	16
	4.3	Machine Learning Algorithm Comparison	16
	4.3.1	Anomaly Detection	17
	4.3.2	Failure Prediction	17
	4.3.3	Root Cause Analysis	17
5		**IMPLEMENTATION AND RESULTS**	**18**
	5.1	Proposed Implementation	18
	5.1.1	Infrastructure and Container Implementation	18
	5.1.2	Monitoring and Telemetry Implementation	18
	5.1.3	AI Engine Implementation	19
	5.1.4	Self-Healing and Incident Management	19
	5.1.5	Dashboard Implementation	19
	5.2	Evaluation and Result Discussion	21
	5.2.1	Live Telemetry Analysis	21
	5.2.2	Incident Recovery Analysis	22
	5.2.3	Results	23
	5.2.4	Test Scenario Summary	23
	5.2.5	Comparison with Manual Recovery	24
6		**CONCLUSION AND FUTURE WORK**	**25**
	6.1	Conclusion	25
	6.2	Future Work	25
		APPENDIX	**27**
		REFERENCES	**28**

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Comparison of infrastructure recovery approaches	8
3.1	Software requirements of the proposed system	10
3.2	Hardware requirements of the proposed system	10
3.3	Telemetry features used by the AI engine	11
3.4	Structure of the incident record	13
4.1	Anomaly detection algorithm comparison	17
4.2	Failure prediction algorithm comparison	17
4.3	Root cause analysis approach comparison	17
5.1	Docker Compose service topology	18
5.2	Live telemetry snapshot	21
5.3	Incident management and healing audit trail	22
5.4	Recovery performance summary	23
5.5	Test scenario summary	23
5.6	Manual recovery versus the proposed framework	24

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Architecture of the self-healing framework	9
3.2	Activity diagram of the self-healing lifecycle	12
3.3	Data flow diagram of the proposed system	12
4.1	Self-healing processing flow	16
5.1	Dashboard header and live self-healing center	19
5.2	System overview and infrastructure simulation	20
5.3	Real-time telemetry and AI engine status	20
5.4	Incident management and healing audit trail	21
5.5	Recovery duration per incident	22

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
AIOps	Artificial Intelligence for IT Operations
API	Application Programming Interface
CPU	Central Processing Unit
DFD	Data Flow Diagram
IT	Information Technology
ML	Machine Learning
MTTD	Mean Time To Detect
MTTR	Mean Time To Recover
PromQL	Prometheus Query Language
RAM	Random Access Memory
RCA	Root Cause Analysis
REST	Representational State Transfer
RX / TX	Received / Transmitted (network traffic)
SRE	Site Reliability Engineering
TSDB	Time-Series Database
UI	User Interface
XGBoost	Extreme Gradient Boosting

CHAPTER 1
INTRODUCTION TO SELF-HEALING IT INFRASTRUCTURE

1.1 GENERAL

Modern digital services are delivered by IT infrastructure made up of many interconnected resources: servers, web front-ends, application programming interfaces (APIs), databases, background workers, containers and the network that links them. These resources must remain available and responsive at all times because users experience any failure as an outage of the whole service. In practice, unexpected events such as resource overload, memory exhaustion, process crashes, failed health checks and network degradation occur regularly, and each of them can interrupt the service if it is not handled quickly.

In conventional infrastructure management, monitoring tools raise an alert and an administrator then investigates the dashboards, metrics and logs, decides what has gone wrong and performs a recovery step such as restarting a service. This manual chain increases the Mean Time To Detect (MTTD) and the Mean Time To Recover (MTTR), particularly when several components depend on each other and the true source of an incident is not immediately clear. It also depends on an engineer being available at the moment the incident occurs.

Self-healing systems aim to shorten this chain by allowing the infrastructure to detect, diagnose and repair common faults automatically. Combined with Artificial Intelligence for IT Operations (AIOps), machine-learning models can learn the normal operating pattern of a system, flag unusual behaviour, estimate the likelihood of failure and point to the probable cause. This project develops an **AI-Based Self-Healing IT Infrastructure Framework** that brings these capabilities together in a working prototype: a containerised microservice environment is monitored continuously, analysed by three machine-learning components, and repaired through controlled and verified recovery actions whose outcomes are recorded and displayed on a live dashboard.

The framework is intended as an operations decision-and-action layer rather than a replacement for engineering judgement. Every automated action is restricted to an allow-listed set of safe operations, every incident is logged with its detected cause, action, outcome and duration, and the post-recovery health check ensures that an action is only reported as successful when the affected service has actually returned to a healthy state.

1.2 OBJECTIVES

The primary objective of the project is to design, implement and evaluate an integrated framework that continuously monitors IT infrastructure, identifies abnormal behaviour and

potential failures using machine learning, and performs safe, verifiable recovery with a complete audit trail. The specific objectives are:

1. **Infrastructure Simulation:** To build a representative, reproducible microservice environment (web server, API server, database and worker) using Docker Compose so that the framework can be tested under controlled conditions.
2. **Continuous Monitoring:** To collect host, container and application telemetry such as CPU and memory utilisation, network traffic, container resource usage, API request rate and latency using Prometheus and its exporters.
3. **Data Processing:** To clean, normalise and transform the collected telemetry into a consistent feature vector that can be consumed by the AI models.
4. **Anomaly Detection:** To detect unusual infrastructure behaviour using the unsupervised Isolation Forest algorithm.
5. **Failure Prediction:** To estimate the risk of an upcoming infrastructure failure using an XGBoost classifier.
6. **Root Cause Analysis:** To identify the probable cause of an incident using a Decision Tree combined with metric and health-check rules.
7. **Controlled Self-Healing:** To execute allow-listed recovery actions, such as restarting a failed container, and to verify that the service has recovered before the incident is closed.
8. **Incident Management and Visualisation:** To store every incident in a PostgreSQL audit trail and present infrastructure health, AI outputs, recovery status and history on a real-time dashboard.

1.3 EXISTING SYSTEM

The baseline for this project is conventional monitoring-based operations. Tools such as Prometheus, Grafana, Nagios or Zabbix collect metrics and raise alerts when values cross a fixed threshold. Container orchestrators provide basic restart policies and liveness probes that restart a process when it exits or stops responding. A typical existing workflow can be represented as:

Infrastructure → Metrics / Logs → Static Threshold Alert → Administrator Investigation → Manual Diagnosis → Manual Recovery → Manual Verification

These systems give good visibility but have several limitations. Static thresholds must be tuned by hand and either miss gradual or multi-metric deviations or generate many false alerts. Monitoring reports that something is wrong but does not predict an upcoming failure or explain why it happened. Simple restart policies act on a single signal without

considering the wider system context, and they do not record a structured explanation of the incident. Finally, diagnosis, recovery and verification remain manual steps that depend on the availability and experience of an administrator, which increases MTTR and makes incident history inconsistent.

1.4 PROPOSED SYSTEM

The proposed system is an AI-Based Self-Healing IT Infrastructure Framework that connects monitoring, analysis and recovery into one closed loop. A Docker Compose environment hosts an Nginx web server, a Python FastAPI API server, a PostgreSQL 15 database and a Python healing worker. Prometheus scrapes telemetry from Node Exporter (host metrics), cAdvisor (container metrics) and a FastAPI instrumentator (request rate and latency).

A data-processing module builds a nine-feature telemetry vector. Isolation Forest flags anomalous behaviour, XGBoost estimates failure risk, and a rule-based Decision Tree identifies the probable root cause, such as high memory usage or an unavailable API or web service. When an incident is confirmed, the self-healing engine follows a **Detect → Analyze → Heal → Verify → Result** lifecycle: it executes an allow-listed container restart through the worker daemon, re-checks the service health, and writes the incident type, AI detection, predicted risk, root cause, healing action, status and recovery duration to the PostgreSQL incident store. A dashboard presents overall system health, live telemetry, AI module outputs and the incident audit trail, with links to Grafana, Prometheus and the API documentation.

1.5 SCOPE OF THE PROJECT

The scope of the project covers a monitored, containerised environment consisting of representative infrastructure components (web, API, database and worker). It includes telemetry collection, data preprocessing, AI-based anomaly detection, failure-risk prediction, root-cause analysis, controlled container-level recovery, post-recovery verification, incident persistence and dashboard visualisation. Recovery actions are deliberately limited to safe, reversible operations such as restarting a container or service. Large-scale production deployment, multi-cluster orchestration, automatic code or configuration rollback, and security-incident response are outside the current scope and are discussed as future work.

1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem domain, objectives, existing approach, proposed system, scope and problem statement. Chapter 2 reviews the literature on self-healing systems, AIOps, anomaly detection, failure prediction, root-cause analysis and container

monitoring, and identifies the research gap. Chapter 3 presents the system architecture, software, hardware and dataset requirements, and the system design through activity and data flow diagrams. Chapter 4 describes the methodology, the eight modules of the framework and the comparison of candidate machine-learning algorithms. Chapter 5 explains the implementation and discusses the results obtained from the running prototype, including live telemetry and the incident recovery audit trail. Chapter 6 concludes the report and outlines future work, followed by the appendix and references.

1.7 PROBLEM STATEMENT

IT services experience abnormal resource usage, application and container failures, network degradation and other operational incidents. When these events are handled through separate monitoring tools and manual troubleshooting, detection, diagnosis and recovery are delayed, the same repetitive recovery steps are performed by hand, and the reasoning behind each action is rarely recorded in a consistent way. Static threshold alerts also produce false alarms or miss combined deviations across several metrics.

The problem addressed by this project is therefore to design an integrated framework that can (i) monitor infrastructure resources and services continuously, (ii) identify abnormal behaviour without relying only on fixed thresholds, (iii) estimate the risk of an upcoming failure, (iv) determine the probable cause of an incident in an interpretable way, (v) perform safe, allow-listed recovery actions automatically, and (vi) verify and record the outcome of every action so that the system remains transparent and auditable.

The key challenges that the framework must handle are:

- **Multi-metric behaviour:** failures often appear as a combination of small changes in CPU, memory, network and latency rather than one metric crossing a limit.
- **Lack of labelled failure data:** real incidents are rare, so detection must work largely without labels.
- **Explainability:** operators must understand why an automated action was taken.
- **Safe automation:** recovery must be restricted to approved actions and must not cause additional disruption.
- **Verification and traceability:** an action is only useful if the service is confirmed healthy afterwards and the incident is recorded.

CHAPTER 2
LITERATURE SURVEY ON SELF-HEALING AND AIOPS

2.1 LITERATURE REVIEW

Research relevant to this project spans autonomic computing, self-healing systems, AIOps, anomaly detection, failure prediction, root-cause analysis and container-based infrastructure monitoring. This section reviews these areas and explains how each influences the design of the proposed framework.

2.1.1 Autonomic Computing and Self-Healing Systems

Kephart and Chess introduced the vision of autonomic computing, in which systems manage themselves according to high-level objectives through self-configuration, self-optimisation, self-protection and self-healing [1]. Their MAPE-K loop (Monitor, Analyse, Plan, Execute over shared Knowledge) remains the reference model for automated operations. Ghosh et al. surveyed self-healing systems and described the common stages of maintaining health, detecting system failure and recovering from failure [2], while Psaier and Dustdar reviewed self-healing approaches and systems across several domains and highlighted the need for reliable detection and safe recovery strategies [3].

Therefore, the proposed framework adopts a closed control loop equivalent to MAPE-K: telemetry is monitored, analysed by AI models, a recovery action is planned from allow-listed rules, executed, and its outcome is verified and stored as knowledge in the incident history.

2.1.2 AIOps and Incident Management

AIOps applies machine learning and data analytics to IT operations data in order to detect, diagnose and resolve problems faster. Dang, Lin and Huang discussed the real-world challenges of AIOps in large cloud services and emphasised that useful AIOps solutions must be integrated into the operational workflow rather than operating as isolated models [4]. Notaro, Cardoso and Gerndt surveyed AIOps methods for failure management and organised them into failure prevention, online failure prediction, failure detection, root-cause analysis and remediation [5]. Chen et al. examined incident management at a large cloud provider and argued for intelligent incident management that connects detection, triage, diagnosis and mitigation [6].

Therefore, the framework is structured as a pipeline that covers the full failure-management lifecycle described in [5], from detection and prediction to diagnosis, remediation and recording, instead of addressing only one stage.

2.1.3 Anomaly Detection in Infrastructure Telemetry

Infrastructure anomalies are often rare, unlabelled and multi-dimensional, which makes unsupervised methods attractive. Liu, Ting and Zhou proposed Isolation Forest, which isolates anomalies by random partitioning; anomalous points require fewer splits to isolate and therefore receive higher anomaly scores [7]. The method has linear time complexity and low memory requirements, making it suitable for continuous scoring of streaming metrics. Soldani and Brogi surveyed anomaly detection and root-cause analysis techniques for service-based cloud applications and showed that metric-based detection is one of the most widely used approaches in microservice systems [8].

Therefore, the proposed framework uses Isolation Forest on the processed telemetry vector so that combined deviations across CPU, memory, network, container and API metrics can be detected without requiring labelled failure data.

2.1.4 Failure Prediction

Failure prediction aims to warn operators before a service actually fails. Lin et al. predicted node failures in cloud service systems by combining spatial and temporal features with machine-learning ranking models, demonstrating that monitoring data contains signals that precede failures [9]. Gradient-boosted decision trees are widely used for such tabular prediction tasks; Chen and Guestrin introduced XGBoost, a scalable tree-boosting system with regularisation and efficient handling of sparse data, which achieves strong accuracy on structured data [10].

Therefore, the framework uses an XGBoost classifier to convert the current telemetry features into a failure-risk level that complements the anomaly signal from Isolation Forest.

2.1.5 Root Cause Analysis

Root-cause analysis (RCA) identifies the component or condition responsible for an incident. Survey work on microservice RCA shows that approaches range from rule-based and statistical methods to graph-based and learning-based techniques [8]. Decision Trees, formalised in the CART framework by Breiman et al. [11], produce interpretable if-then rules that can be read and validated by operators, which is valuable when an automated system must justify why it is taking a recovery action.

Therefore, the framework combines a Decision Tree with explicit metric and health-check rules so that each incident is labelled with a human-readable cause such as HIGH_MEMORY, API Server Service Unavailable or Web Server Down.

2.1.6 Container Platforms and Monitoring

Containers package applications with their dependencies and allow services to be restarted quickly and consistently. Burns et al. described the lessons learned from Borg, Omega and Kubernetes, including the value of declarative management and automatic restart of failed containers [12]. Prometheus provides a pull-based monitoring system with a time-series database and the PromQL query language [13], and Docker Compose allows a multi-container application to be defined and run from a single configuration file [14]. Site Reliability Engineering practice further stresses monitoring of latency, traffic, errors and saturation, and the use of automation to reduce repetitive operational work [15].

Therefore, the prototype uses Docker Compose for a reproducible environment, Prometheus with Node Exporter and cAdvisor for telemetry, and container restart as the primary recovery action.

2.1.7 Research Gap

The reviewed literature demonstrates progress in individual areas such as autonomic and self-healing systems [1]–[3], AIOps failure management [4]–[6], unsupervised anomaly detection [7], [8], failure prediction [9], [10], interpretable root-cause analysis [11] and container monitoring [12]–[15]. However, these capabilities are frequently studied or deployed separately: monitoring tools detect but do not diagnose, orchestrator restart policies recover but do not explain, and many AIOps models produce predictions without closing the loop through verified remediation. There is a need for an integrated, lightweight framework that combines detection, prediction, diagnosis, controlled recovery, post-recovery verification and auditable incident history in a single workflow. The contribution of this project lies in that integration and in its implementation as a working prototype, rather than in claiming that any individual technique is new.

2.2 COMPARISON AND DISCUSSION

The reviewed approaches reveal several practical challenges. First, fixed thresholds on single metrics cannot represent the normal behaviour of a system with many interacting components; the framework therefore uses a multi-feature anomaly model. Second, detecting an anomaly is not the same as predicting that a service will fail, so an independent risk estimate is required before high-impact actions are taken. Third, an automated action must be explainable, which motivates an interpretable root-cause step. Fourth, an action is only useful if it is verified; the framework re-checks health after every recovery and records the measured duration. Table 2.1 compares the proposed framework with common existing approaches.

Table 2.1. Comparison of infrastructure recovery approaches

Feature	Threshold Monitoring (e.g., Nagios, Zabbix)	Orchestrator Restart Policies (e.g., Docker, Kubernetes)	Proposed Framework
Detection	Static per-metric thresholds	Process exit or liveness probe failure	Isolation Forest on 9 telemetry features plus health checks
Failure prediction	Not available	Not available	XGBoost failure-risk classification
Root cause	Manual investigation	Not provided	Decision Tree + metric/health rules
Recovery	Manual	Automatic restart on a single signal	Allow-listed restart driven by combined AI signals
Verification	Manual	Implicit (probe passes)	Explicit health re-check with measured duration
Audit trail	Alert history only	Container events only	Structured PostgreSQL incident record
Visibility	Metric dashboards	CLI / orchestrator events	Unified dashboard: health, telemetry, AI outputs, incidents

2.3 CONCLUSION

The literature shows that infrastructure operations have progressed from manual monitoring toward autonomic management and AIOps, with mature techniques available for anomaly detection, failure prediction and root-cause analysis. However, these techniques are often applied separately, leaving gaps between detecting an issue, understanding it and safely resolving it. The proposed framework addresses this gap by combining continuous telemetry, Isolation Forest, XGBoost, a rule-based Decision Tree, controlled container recovery, verification and incident persistence in one closed loop. This forms the basis for the system design, implementation and evaluation presented in the following chapters.

CHAPTER 3
PROPOSED SYSTEM - SELF-HEALING FRAMEWORK

3.1 SYSTEM ARCHITECTURE

The proposed architecture is organised into five tiers: infrastructure simulation, monitoring and telemetry, AI engine, controlled recovery, and incident management and presentation. Separating these concerns allows the monitored services, the AI models and the recovery logic to evolve independently while sharing a common data flow. Telemetry flows downward from the containers to Prometheus and the AI engine, decisions flow into the recovery tier, and recovery actions flow back to the infrastructure tier, closing the self-healing loop as shown in Figure 3.1.

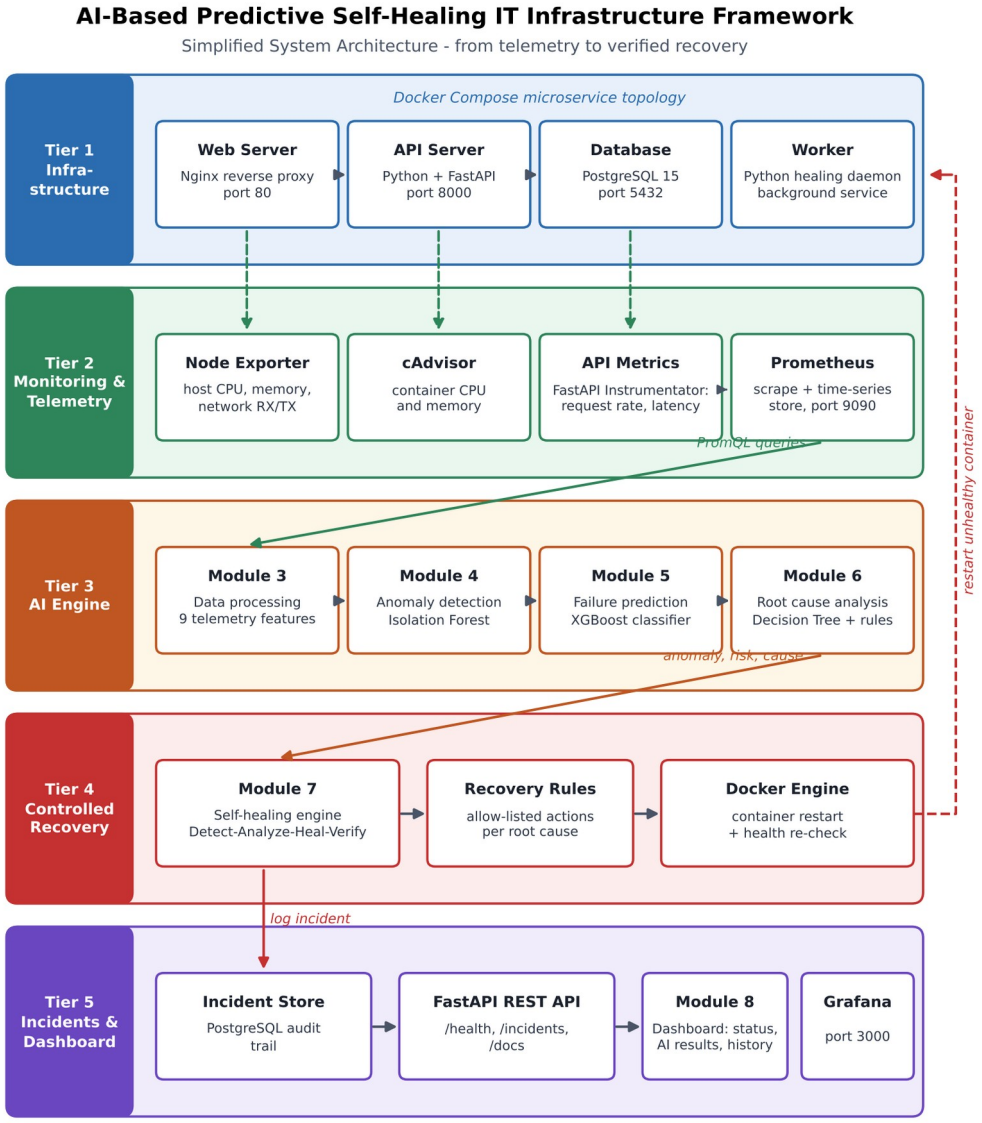


Figure 3.1. Architecture of the self-healing framework

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.1. Software requirements of the proposed system

Software Component	Technology	Purpose
Containerisation	Docker Engine, Docker Compose	Define, run and restart the microservice environment
Web Server	Nginx	Reverse proxy and web entry point (port 80)
API Server	Python, FastAPI	Application service, health endpoint, REST APIs and /docs (port 8000)
Database	PostgreSQL 15	Application data and incident audit trail (port 5432)
Worker	Python healing daemon	Background monitoring loop and recovery execution
Monitoring	Prometheus, Node Exporter, cAdvisor, FastAPI Instrumentator	Collection and storage of host, container and API metrics (port 9090)
Visualisation	Grafana, custom web dashboard	Metric dashboards (port 3000) and self-healing dashboard
Machine Learning	scikit-learn (Isolation Forest, Decision Tree), XGBoost	Anomaly detection, failure prediction and root-cause analysis
Data Handling	pandas, NumPy	Feature extraction, cleaning and normalisation

3.2.2 Hardware Requirements

Table 3.2. Hardware requirements of the proposed system

Component	Suggested Requirement	Purpose
CPU	Modern multi-core processor (4 cores or more)	Running containers, monitoring stack and model inference
RAM	8 GB minimum (the test host reported about 7.9 GB)	Concurrent containers, Prometheus and AI engine
Storage	SSD with at least 20 GB free	Container images, Prometheus time series, database and models
Network	Local Docker bridge network	Communication between services and exporters

3.2.3 Dataset and Feature Requirements

The AI engine is driven by time-series telemetry scraped by Prometheus rather than by a static external dataset. At each analysis cycle, the data-processing module queries

Prometheus and extracts the nine telemetry metrics reported on the dashboard. These include the host, container and application indicators listed in Table 3.3, together with the service health status obtained from health checks. Values are cleaned for missing or invalid samples, converted to consistent units and normalised before being passed to the models.

Table 3.3. Telemetry features used by the AI engine

Feature	Source	Unit
Host CPU utilisation	Node Exporter	%
Host memory utilisation	Node Exporter	%
Network inbound traffic	Node Exporter (RX)	Bytes/s
Network outbound traffic	Node Exporter (TX)	Bytes/s
Container CPU usage	cAdvisor (Docker stats)	%
Container memory usage	cAdvisor	MB
API request rate	FastAPI Instrumentator	requests/s
API average latency	FastAPI Instrumentator	ms

For supervised components (XGBoost and the Decision Tree), training samples are labelled from normal operating periods and from controlled fault-injection runs in which services are stopped or stressed. The final report of the trained models should state the number of samples, the ratio of normal to failure samples, the train/test split and how labels were produced, so that the reported performance can be interpreted correctly.

3.3 SYSTEM DESIGN

3.3.1 Activity Diagram

Figure 3.2 shows the activity flow of one monitoring and healing cycle. If no anomaly or service failure is detected, the system returns to monitoring. Otherwise the incident passes through prediction, root-cause analysis and recovery, and is only marked as recovered after the health check succeeds.

1. The environment is started with Docker Compose and Prometheus begins scraping Node Exporter, cAdvisor and FastAPI metrics.
2. Module 3 queries the latest values, cleans and normalises them, and builds the nine-feature vector.
3. Module 4 scores the vector with Isolation Forest; together with the service health checks, this decides whether an incident exists.
4. If an incident exists, Module 5 predicts the failure risk with XGBoost and Module 6 labels the root cause with the Decision Tree and rules.

5. Module 7 selects the allow-listed action for the cause and restarts the affected container.
6. The health check is repeated; on success the incident is recorded as RECOVERED with its duration, otherwise the action is retried or escalated.
7. Module 8 refreshes the dashboard and the loop continues monitoring.

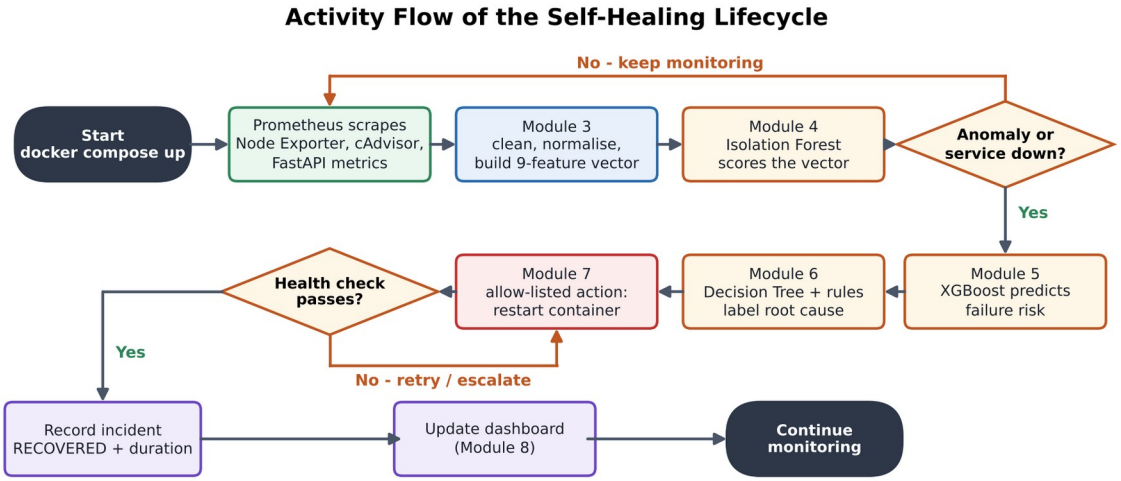


Figure 3.2. Activity diagram of the self-healing lifecycle

3.3.2 Data Flow Diagram

At the context level, the Docker services provide raw telemetry and the administrator receives system status, alerts and incident history. At the first decomposition level, the main processes are metric collection, feature processing, detection and prediction, root-cause analysis, healing and verification, and recording and display. The data stores are the Prometheus time-series database, the trained models and the PostgreSQL incident database, as shown in Figure 3.3.

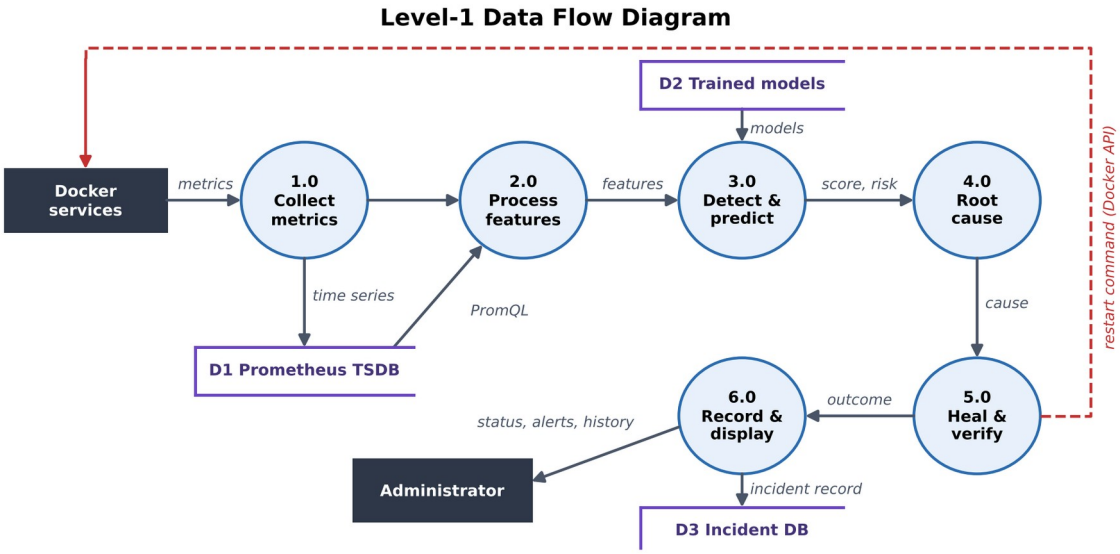


Figure 3.3. Data flow diagram of the proposed system

3.3.3 Incident Record Design

Every incident handled by the framework is stored as one row in the PostgreSQL incidents table. The structure, shown in Table 3.4, follows the columns displayed in the Module 8 audit trail, so that each record captures what was detected, what the AI engine concluded, what action was taken and whether it succeeded.

Table 3.4. Structure of the incident record

Field	Description	Example
id	Unique incident identifier	#11
timestamp	Date and time the incident was detected	2026-09-09 02:41:42
target_service	Container or service affected	self-healing-api
incident_type	How the incident was raised	AUTOMATED_HEALTH_MONITOR
ai_detection	Isolation Forest result	ANOMALY
predicted_risk	XGBoost failure-risk level	HIGH
root_cause	Decision Tree / rule-based cause label	API Server Service Unavailable
healing_action	Recovery action executed	Automated container restart by worker daemon
status	Outcome after verification	RECOVERED
duration	Time from action start to verified recovery	4.18 s

CHAPTER 4
PROJECT DESCRIPTION

4.1 METHODOLOGY

The framework follows a closed-loop methodology based on the Monitor-Analyse-Plan-Execute model of autonomic computing. The process begins by starting the monitored environment with Docker Compose, which brings up the web server, API server, database, worker and monitoring stack on a shared network. Prometheus scrapes Node Exporter, cAdvisor and the FastAPI instrumentator at regular intervals and stores the results as time series.

In each analysis cycle, the data-processing module queries Prometheus, validates the returned samples and builds a normalised nine-feature vector. This vector is scored by the Isolation Forest model, which outputs whether the current state is NORMAL or ANOMALY. The same features are passed to the XGBoost classifier, which estimates a failure-risk level (for example LOW or HIGH). The anomaly result, risk level, raw metrics and service health checks are then combined by the rule-based Decision Tree to label the probable root cause, such as HIGH_MEMORY, API Server Service Unavailable or Web Server Down.

The self-healing engine maps the root cause to a permitted recovery action using an allow-list. For a failed or unavailable service, the worker daemon restarts the corresponding container through the Docker Engine. After the action, the engine polls the service health endpoint and container state until the service is healthy or a time limit is reached. The incident is then written to the PostgreSQL audit trail with its timestamp, target service, incident type, AI detection, predicted risk, root cause, healing action, final status and recovery duration, and the dashboard is refreshed. Separating detection, prediction and diagnosis in this way means that a single noisy signal does not trigger recovery on its own, which reduces unnecessary restarts.

4.2 MAIN MODULES OF THE PROJECT

4.2.1 Infrastructure Simulation

This module creates the environment to be monitored. Docker Compose defines four application services: an Nginx reverse proxy on port 80, a Python FastAPI API server on port 8000, a PostgreSQL 15 database on port 5432 and a Python healing worker running as a background service. These components represent a typical three-tier web application with an operations agent, and can be stopped or stressed to simulate failures in a repeatable way.

4.2.2 Monitoring and Data Collection

Prometheus collects CPU, memory and network metrics from Node Exporter, per-container CPU and memory usage from cAdvisor, and request rate and request duration from the FastAPI instrumentator. The worker also performs direct health checks against the services. All metrics are timestamped and stored in the Prometheus time-series database, and are available through PromQL queries and Grafana dashboards.

4.2.3 Data Processing

The data-processing module retrieves the latest metric values, removes missing or invalid samples, converts byte and time values into consistent units, and scales the features so that they are comparable. The output is a fixed-order nine-feature vector used by all downstream models. The dashboard reports this module as PIPELINE READY when features are being extracted successfully.

4.2.4 AI Anomaly Detection

Isolation Forest is trained on telemetry collected during normal operation. Because anomalies are isolated with fewer random splits, the model assigns a higher anomaly score to unusual combinations of metrics. The module returns NORMAL or ANOMALY together with the score, allowing multi-metric deviations to be detected without hand-tuned thresholds.

4.2.5 Failure Prediction

The XGBoost classifier is trained on labelled telemetry from normal and failure-injection periods. It outputs a failure-risk level that indicates how likely the current state is to lead to a service failure. Using a prediction model alongside anomaly detection allows the system to distinguish harmless deviations from states that require intervention.

4.2.6 Root Cause Analysis

This module combines a Decision Tree with explicit rules on metrics and health checks. The Decision Tree learns which feature conditions correspond to known causes, while the rules capture definite conditions such as a failed health endpoint. The result is an interpretable cause label, for example HIGH_MEMORY, API Server Service Unavailable or Web Server Down, which is shown on the dashboard and stored with the incident.

4.2.7 Self-Healing and Recovery

The recovery module is executed by the worker daemon. It selects an allow-listed action for the identified cause, such as an automated container restart, executes it through the Docker Engine, and then verifies recovery by re-checking the service health. The lifecycle is displayed in the Live Self-Healing Center as five stages: Detect, Analyze, Heal, Verify and

Result. If verification fails, the incident remains open for retry or escalation rather than being reported as recovered.

4.2.8 Dashboard and Incident Management

The dashboard displays overall health, the number of total, healthy and unhealthy services, failure risk, anomaly state and root cause, the status of each simulated service, live telemetry, the state of each AI module and the incident audit trail retrieved from PostgreSQL. It also links to Grafana, Prometheus, the FastAPI interactive documentation and the API health endpoint. The overall processing flow is summarised in Figure 4.1.

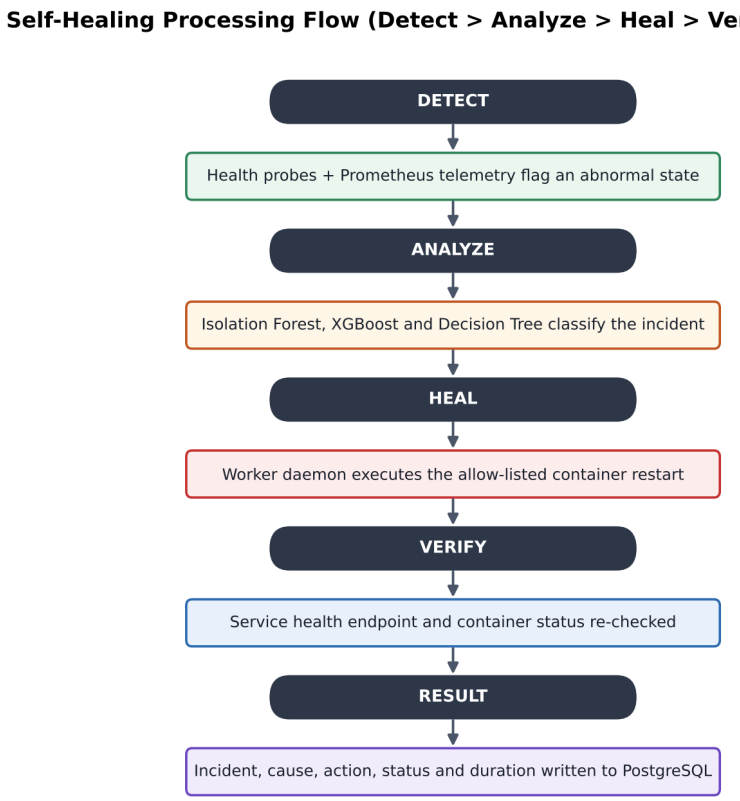


Figure 4.1. Self-healing processing flow

4.3 MACHINE LEARNING ALGORITHM COMPARISON

Candidate algorithms for each AI task were compared on their suitability for continuous infrastructure telemetry: the need for labelled data, the cost of inference, interpretability and robustness. The selected algorithm for each task is marked in the tables below. Quantitative scores (precision, recall, F1-score and false-positive rate) should be added from the project's own evaluation runs; no assumed values are reported here.

4.3.1 Anomaly Detection

Table 4.1. Anomaly detection algorithm comparison

Algorithm	Labels Needed	Strengths	Limitations
Static thresholds	No	Simple, transparent	Single-metric; manual tuning; frequent false alerts
Local Outlier Factor	No	Detects local density outliers	Costly on large or streaming data; sensitive to k
One-Class SVM	No	Flexible decision boundary	Sensitive to kernel and scaling; slower training
Isolation Forest (selected)	No	Fast, linear time; multi-feature; low memory	Score threshold must be chosen; may miss subtle drift

4.3.2 Failure Prediction

Table 4.2. Failure prediction algorithm comparison

Algorithm	Strengths	Limitations
Logistic Regression	Simple, interpretable baseline	Limited to linear relationships between metrics
Random Forest	Handles non-linear patterns; robust to noise	Larger models; less accurate than boosting on many tabular tasks
XGBoost (selected)	High accuracy on tabular data; regularisation; handles imbalance through weighting	Requires labelled failure data and hyper-parameter tuning

4.3.3 Root Cause Analysis

Table 4.3. Root cause analysis approach comparison

Approach	Strengths	Limitations
Manual investigation	Uses expert knowledge	Slow; inconsistent; depends on staff availability
Graph / causal inference	Can trace propagation across many services	Needs dependency graphs and more data; complex
Decision Tree + rules (selected)	Interpretable if-then logic; fast; easy to validate	Covers known incident types; must be extended for new causes

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 PROPOSED IMPLEMENTATION

The framework is implemented as a set of Docker containers orchestrated with Docker Compose, a monitoring stack built on Prometheus, an AI engine written in Python, a healing worker daemon and a web dashboard. All components run on a single host and communicate over a Docker network, which makes the environment reproducible and allows failures to be injected safely.

5.1.1 Infrastructure and Container Implementation

The application services are defined in a Docker Compose file. Each service is given a fixed name, port mapping and health check so that the worker can reliably identify and restart it. Table 5.1 lists the service topology used in the prototype.

Table 5.1. Docker Compose service topology

Service	Technology	Port	Role
Web Server (self-healing-web)	Nginx reverse proxy	80	Entry point that forwards requests to the API
API Server (self-healing-api)	Python + FastAPI	8000	Application logic, /health and REST endpoints, /docs
Database	PostgreSQL 15	5432	Application data and incident records
Worker	Python healing daemon	-	Monitoring loop, AI calls and recovery execution
Prometheus	Prometheus server	9090	Metric scraping and time-series storage
Grafana	Grafana	3000	Metric visualisation
Exporters	Node Exporter, cAdvisor	-	Host and container metrics

5.1.2 Monitoring and Telemetry Implementation

Prometheus is configured with scrape jobs for Node Exporter, cAdvisor and the FastAPI service. Node Exporter supplies host CPU load, memory usage and network RX/TX bytes; cAdvisor supplies per-container CPU and memory (Docker stats); and the FastAPI instrumentator exposes the request rate and average request duration. The dashboard shows these values in the Module 2 Real-Time Infrastructure Telemetry panel, updated from the Prometheus metrics stream.

5.1.3 AI Engine Implementation

The AI engine queries Prometheus through its HTTP API using PromQL, assembles the nine telemetry features and applies the stored preprocessing parameters. The Isolation Forest and Decision Tree are implemented with scikit-learn and the failure classifier with the XGBoost library; trained models are loaded once at start-up and reused for every cycle. Each module exposes its result to the dashboard: Module 3 reports the number of extracted features and pipeline status, Module 4 the anomaly state, Module 5 the failure-risk level and Module 6 the identified root cause.

5.1.4 Self-Healing and Incident Management

The worker daemon runs continuously. When a target service fails its health check or the AI engine reports an incident, the worker records the start time, restarts the affected container through the Docker Engine, and polls the service until it is healthy again. The measured recovery duration and all incident attributes are written to the PostgreSQL incidents table. The incident type used for these automatically detected events is AUTOMATED_HEALTH_MONITOR, and the healing action is recorded as an automated container restart executed by the worker daemon.

5.1.5 Dashboard Implementation

The dashboard is a web interface backed by the FastAPI REST endpoints. It refreshes periodically and is divided into the header and Live Self-Healing Center (Figure 5.1), the System Overview and Module 1 service cards (Figure 5.2), the Module 2 telemetry panel and Modules 3-7 AI engine cards (Figure 5.3), and the Module 8 incident audit trail (Figure 5.4).

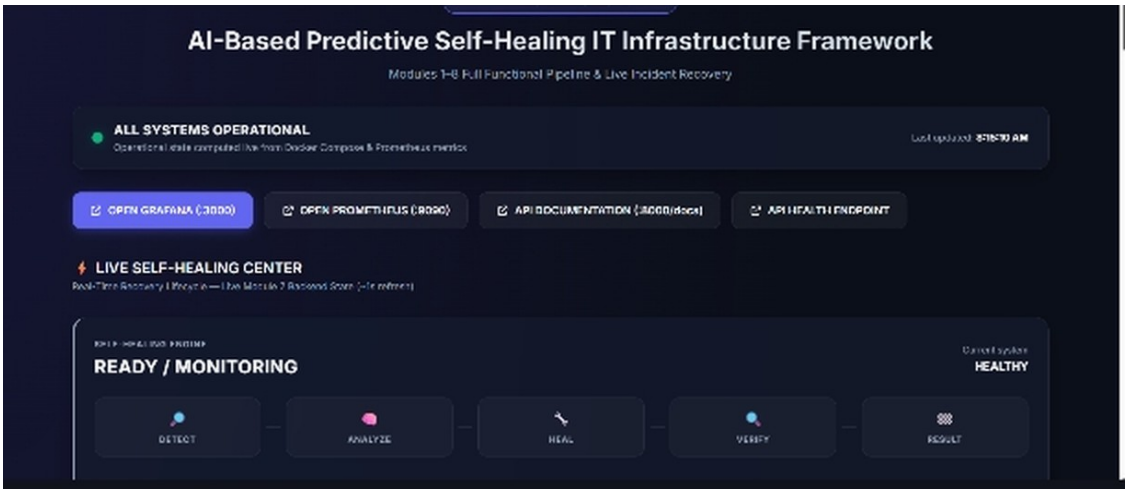


Figure 5.1. Dashboard header and live self-healing center

The header reports ALL SYSTEMS OPERATIONAL with state computed live from Docker Compose and Prometheus metrics, and provides shortcuts to Grafana (3000), Prometheus (9090), the API documentation (8000/docs) and the API health endpoint. The

self-healing engine is in the READY / MONITORING state with the current system HEALTHY, and the five lifecycle stages Detect, Analyze, Heal, Verify and Result are shown.

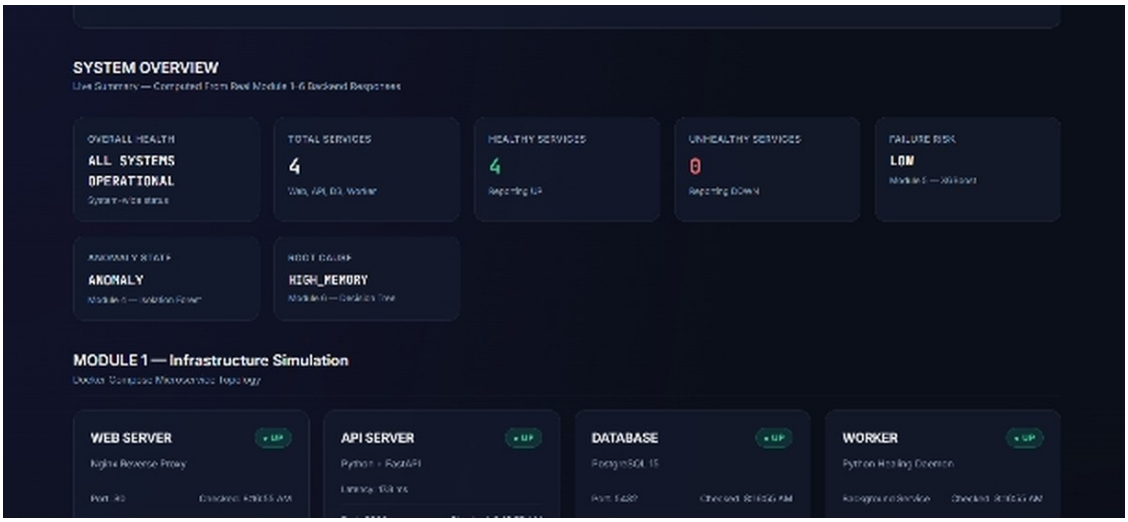


Figure 5.2. System overview and infrastructure simulation

The system overview shows four total services (Web, API, DB and Worker), four healthy and zero unhealthy. The failure risk from Module 5 (XGBoost) is LOW, the anomaly state from Module 4 (Isolation Forest) is ANOMALY and the root cause from Module 6 (Decision Tree) is HIGH_MEMORY. The Module 1 cards confirm that the web server, API server, database and worker are all UP.

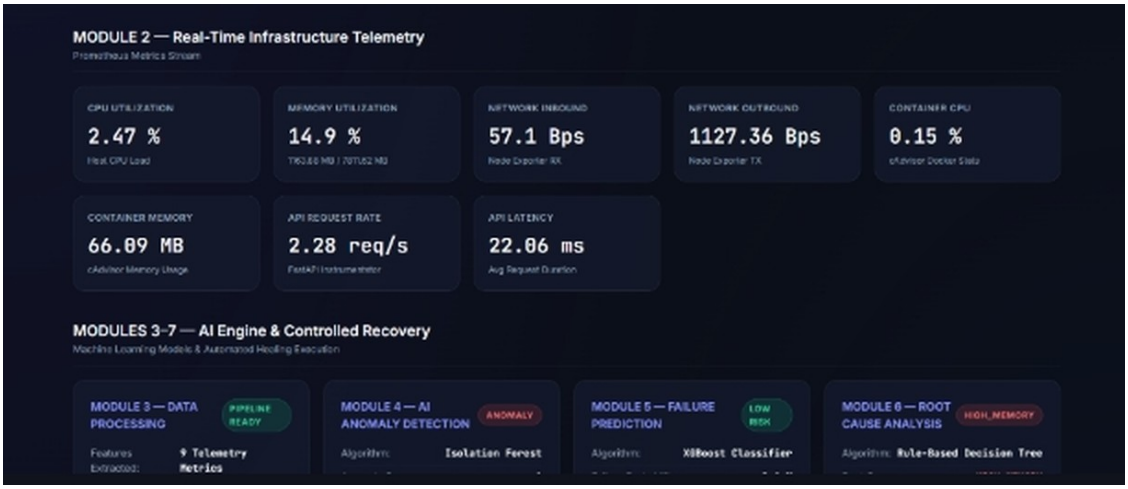


Figure 5.3. Real-time telemetry and AI engine status

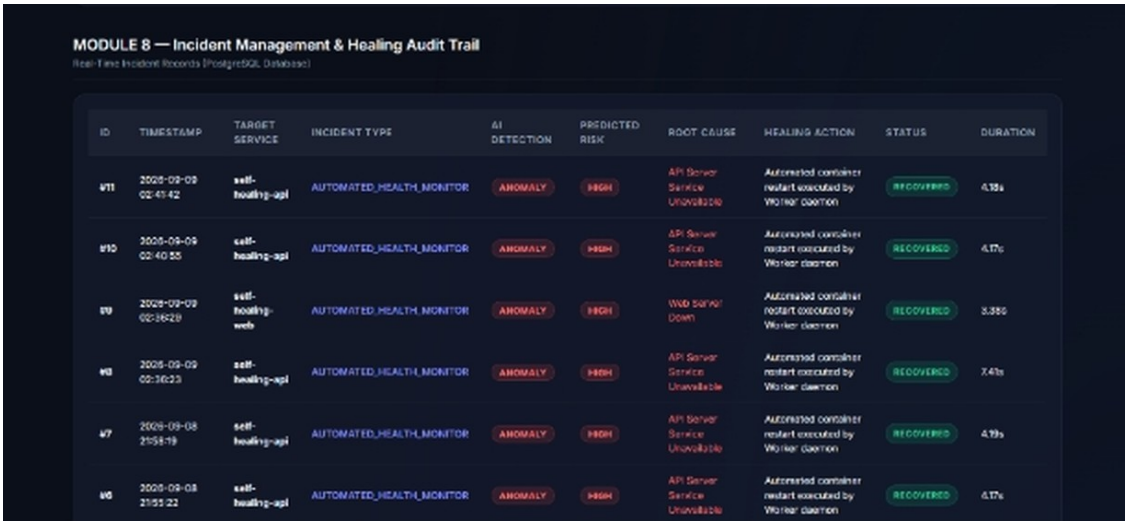


Figure 5.4. Incident management and healing audit trail

5.2 EVALUATION AND RESULT DISCUSSION

The prototype was evaluated by running the full stack, observing live telemetry and injecting service failures so that the self-healing loop was exercised end to end. The results below are taken directly from the running dashboard and its PostgreSQL incident records.

5.2.1 Live Telemetry Analysis

Table 5.2. Live telemetry snapshot

Metric	Observed Value	Source
CPU utilisation	2.47 %	Node Exporter (host CPU load)
Memory utilisation	14.9 %	Node Exporter
Network inbound	57.1 Bps	Node Exporter RX
Network outbound	1127.36 Bps	Node Exporter TX
Container CPU	0.15 %	cAdvisor Docker stats
Container memory	66.09 MB	cAdvisor memory usage
API request rate	2.28 req/s	FastAPI instrumentator
API latency	22.06 ms	Average request duration

The snapshot shows that the monitoring overhead is low: the host CPU load is 2.47 % and the application container uses only 0.15 % CPU and 66.09 MB of memory, while the API serves 2.28 requests per second with an average latency of 22.06 ms. This confirms that continuous telemetry collection and AI scoring can run alongside the monitored services without significantly affecting their performance.

At the same moment the Isolation Forest reported an ANOMALY with the root cause HIGH_MEMORY, while XGBoost predicted LOW failure risk and all four services

remained healthy. This illustrates the layered decision design: an unusual pattern was flagged and explained for the operator, but because the predicted failure risk was low and no service was down, no disruptive recovery action was triggered.

5.2.2 Incident Recovery Analysis

Table 5.3. Incident management and healing audit trail

ID	Timestamp	Target Service	AI Detection / Risk	Root Cause	Status	Time
#11	2026-09-09 02:41:42	self-healing-api	ANOMALY / HIGH	API Server Service Unavailable	RECOVERED	4.18 s
#10	2026-09-09 02:40:55	self-healing-api	ANOMALY / HIGH	API Server Service Unavailable	RECOVERED	4.17 s
#9	2026-09-09 02:38:29	self-healing-web	ANOMALY / HIGH	Web Server Down	RECOVERED	3.38 s
#8	2026-09-09 02:36:23	self-healing-api	ANOMALY / HIGH	API Server Service Unavailable	RECOVERED	7.41 s
#7	2026-09-08 21:58:19	self-healing-api	ANOMALY / HIGH	API Server Service Unavailable	RECOVERED	4.19 s
#6	2026-09-08 21:55:22	self-healing-api	ANOMALY / HIGH	API Server Service Unavailable	RECOVERED	4.17 s

All incidents were of type AUTOMATED_HEALTH_MONITOR and the healing action in every case was an automated container restart executed by the worker daemon. Figure 5.5 compares the recovery duration of each incident.

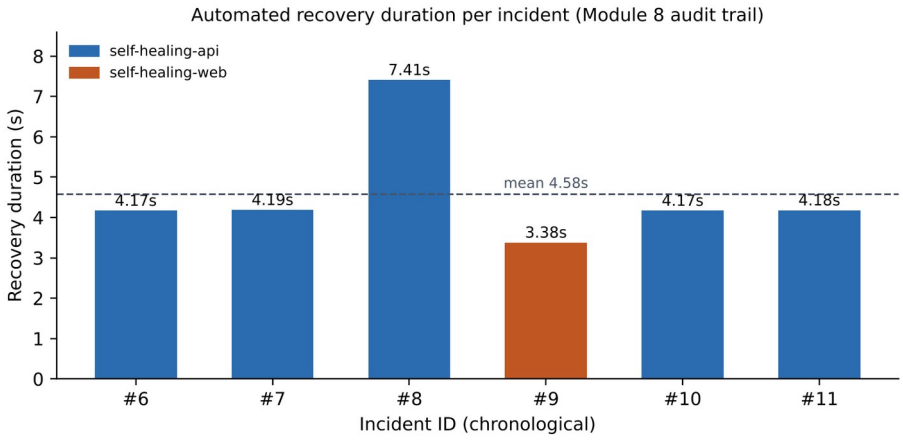


Figure 5.5. Recovery duration per incident

5.2.3 Results

Table 5.4. Recovery performance summary

Measure	Value
Incidents recorded in the audit trail (#6 - #11)	6
Incidents automatically recovered	6 of 6 (100 %)
Services affected	API server: 5, Web server: 1
Mean recovery duration	4.58 s
Median recovery duration	4.18 s
Fastest / slowest recovery	3.38 s / 7.41 s
Manual interventions required	0

Every incident in the recorded audit trail was detected as an ANOMALY with HIGH predicted risk, correctly labelled with the failed service as root cause, automatically restarted and verified as RECOVERED. The typical recovery time was about four seconds, with a mean of 4.58 s and a median of 4.18 s. The single slower case (incident #8, 7.41 s) still recovered without intervention; longer start-up time of the API container on that run is the most likely reason and can be confirmed from the container logs. The web-server incident recovered fastest (3.38 s), which is consistent with Nginx starting more quickly than the Python API service.

Compared with a manual process, in which an operator must notice the alert, investigate and restart the service, the framework closes the full detect-to-verify loop in seconds and records a consistent explanation for every event. These results are based on six controlled failure injections in a single-host environment; larger test campaigns with more failure types, repeated runs and formal measurement of detection precision, recall, false-positive rate and prediction lead time are needed to generalise the findings.

5.2.4 Test Scenario Summary

Table 5.5 summarises the test scenarios exercised during evaluation and the behaviour observed on the dashboard for each of them.

Table 5.5. Test scenario summary

Scenario	Expected Behaviour	Observed Result
Normal operation, all services up	No incident; engine stays in READY / MONITORING	All 4 services UP; system HEALTHY
API container stopped or unavailable	Root cause API Server Service Unavailable; container restarted and verified	5 incidents (#6, #7, #8, #10, #11) RECOVERED in 4.17 - 7.41 s

Scenario	Expected Behaviour	Observed Result
Web (Nginx) container stopped	Root cause Web Server Down; container restarted and verified	Incident #9 RECOVERED in 3.38 s
Unusual memory pattern, services healthy	Anomaly flagged and explained; no restart while risk is LOW	ANOMALY with root cause HIGH_MEMORY, risk LOW, no disruptive action

The scenarios confirm that the framework acts only when a service is actually failing or the predicted risk is high, and that it remains passive but informative when it detects a deviation that does not yet threaten availability.

5.2.5 Comparison with Manual Recovery

Table 5.6 compares each stage of incident handling in a conventional manual process with the behaviour of the implemented framework, based on the incidents recorded during evaluation.

Table 5.6. Manual recovery versus the proposed framework

Stage	Manual Process	Proposed Framework
Detection	Operator notices an alert or a user complaint	Continuous health checks and Isolation Forest scoring
Risk assessment	Based on operator experience	XGBoost failure-risk level (LOW / HIGH)
Diagnosis	Manual inspection of dashboards and logs	Decision Tree + rules give a labelled root cause
Recovery action	Operator restarts the service by hand	Worker daemon restarts the container automatically
Verification	Operator re-checks the service manually	Automatic health re-check before closing the incident
Recovery time	Depends on operator availability	3.38 - 7.41 s per incident (mean 4.58 s) in testing
Documentation	Often incomplete or inconsistent	Structured PostgreSQL record for every incident

The comparison shows that the main benefit of the framework is not only speed but consistency: every incident follows the same detect, analyse, heal and verify sequence and leaves the same structured evidence, which simplifies later review and improvement of the system.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This report presented an AI-Based Self-Healing IT Infrastructure Framework that integrates continuous monitoring, machine-learning analysis and controlled recovery into a single closed loop. A representative microservice environment consisting of Nginx, FastAPI, PostgreSQL and a Python healing worker was deployed with Docker Compose and monitored by Prometheus using Node Exporter, cAdvisor and a FastAPI instrumentator. Telemetry is converted into a nine-feature vector that is analysed by Isolation Forest for anomaly detection, XGBoost for failure-risk prediction and a rule-based Decision Tree for root-cause analysis.

When an incident is confirmed, the self-healing engine follows a Detect, Analyze, Heal, Verify and Result lifecycle, executes an allow-listed container restart, verifies the service health and stores a complete incident record in PostgreSQL. In the recorded evaluation, all six injected failures of the API and web services were automatically recovered with a mean recovery duration of 4.58 seconds and no manual intervention, while the monitoring stack itself consumed very little CPU and memory. The layered design also showed that an anomaly can be flagged and explained without triggering disruptive action when the predicted failure risk is low. Overall, the project demonstrates that combining AIOps techniques with safe, verified automation can reduce recovery time and manual effort while keeping every automated action transparent and auditable.

6.2 FUTURE WORK

Future development of the framework can extend the current architecture in several directions:

6.2.1 Kubernetes and Multi-Node Deployment

The framework can be extended from a single Docker host to Kubernetes clusters, using the orchestrator API for recovery actions such as rescheduling pods, scaling deployments and draining unhealthy nodes, and evaluating the approach across multiple nodes and services.

6.2.2 Richer Recovery Actions and Approval Policies

Additional actions such as horizontal scaling, cache clearing, configuration rollback and traffic redirection can be added. High-impact actions should require approval or escalation, with automatic rollback when verification fails.

6.2.3 Log Analytics and Large Language Models

Structured log parsing and language-model-based summarisation can enrich root-cause analysis, generate human-readable incident reports and suggest remediation steps for incident types that are not yet covered by the rule set.

6.2.4 Model Monitoring and Continuous Retraining

Concept-drift detection, periodic retraining with newly labelled incidents, and systematic evaluation using precision, recall, false-positive rate, prediction lead time, MTTD and MTTR will improve the reliability of the AI components over time.

6.2.5 Alerting and Ticketing Integration

Integration with notification channels such as email, Slack or Microsoft Teams, and with incident-ticketing systems, will allow the operations team to be informed of every automated action and to track unresolved incidents.

6.2.6 Security and Access Control

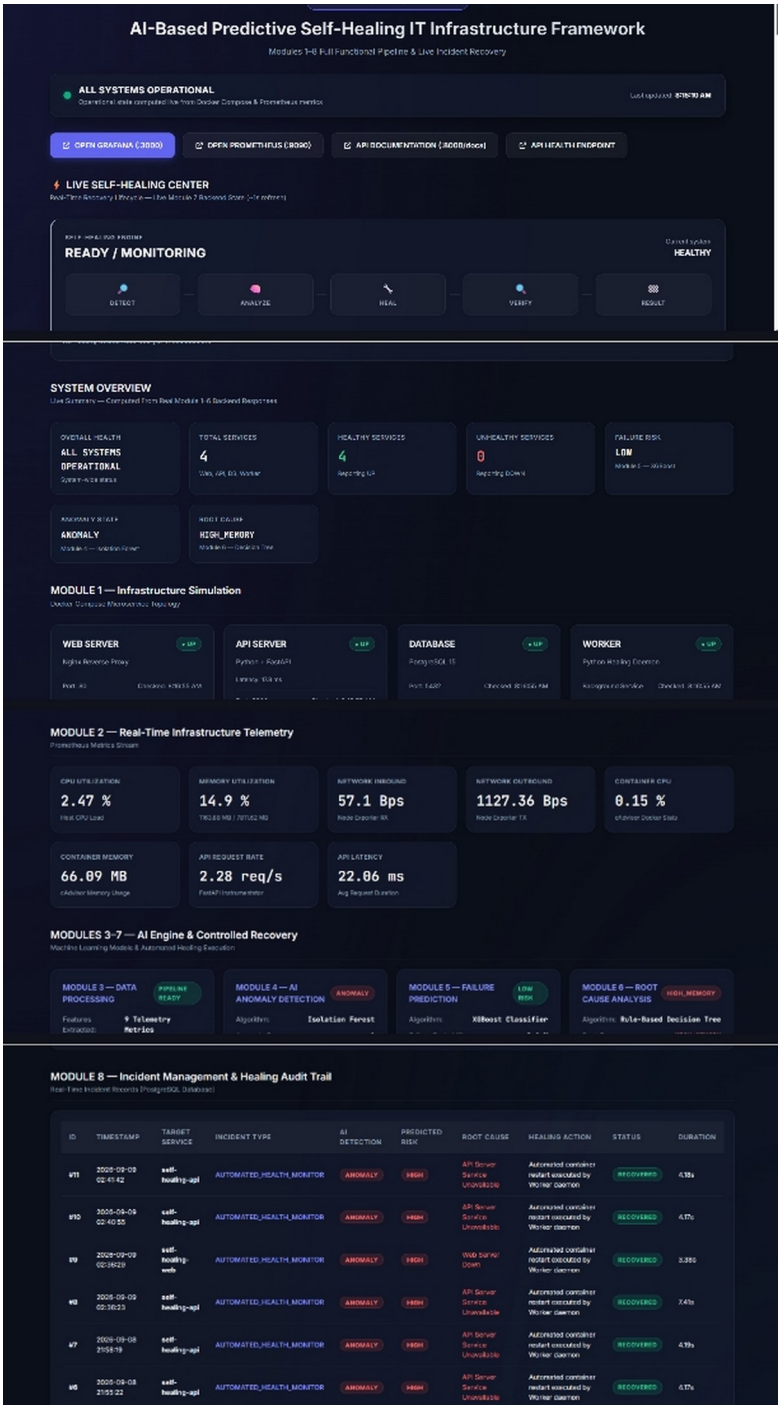
Because the worker can restart containers, access to the Docker API and the recovery endpoints should be protected with authentication, least-privilege service accounts and audit logging of every privileged call. Role-based access to the dashboard and signed configuration for the recovery allow-list will further reduce the risk of misuse.

6.2.7 Hybrid and Multi-Cloud Environments

Finally, the framework can be generalised to monitor services running across on-premises servers and public cloud platforms, using a common telemetry model so that the same AI engine and recovery policies can be applied regardless of where a service is hosted.

APPENDIX : SAMPLE UI SCREENSHOTS

The complete self-healing dashboard captured during the evaluation run is shown below, from the header and Live Self-Healing Center at the top to the Module 8 incident audit trail at the bottom.



Complete self-healing dashboard (Modules 1-8)

REFERENCES

[1] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," IEEE Computer, vol. 36, no. 1, pp. 41-50, 2003, doi: 10.1109/MC.2003.1160055.

[2] D. Ghosh, R. Sharman, H. R. Rao, and S. Upadhyaya, "Self-healing systems - survey and synthesis," Decision Support Systems, vol. 42, no. 4, pp. 2164-2185, 2007, doi: 10.1016/j.dss.2006.06.011.

[3] H. Psaier and S. Dustdar, "A survey on self-healing systems: approaches and systems," Computing, vol. 91, no. 1, pp. 43-73, 2011, doi: 10.1007/s00607-010-0107-y.

[4] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," in Proc. IEEE/ACM 41st Int. Conf. Software Engineering: Companion (ICSE-Companion), 2019, pp. 4-5.

[5] P. Notaro, J. Cardoso, and M. Gerndt, "A Survey of AIOps Methods for Failure Management," ACM Transactions on Intelligent Systems and Technology, vol. 12, no. 6, 2021, doi: 10.1145/3483424.

[6] J. Chen et al., "Towards Intelligent Incident Management: Why We Need It and How We Make It," in Proc. 28th ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1487-1497.

[7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413-422, doi: 10.1109/ICDM.2008.17.

[8] J. Soldani and A. Brogi, "Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey," ACM Computing Surveys, vol. 55, no. 3, 2022, doi: 10.1145/3501297.

[9] Q. Lin et al., "Predicting Node Failure in Cloud Service Systems," in Proc. 26th ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE), 2018, pp. 480-490.

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[11] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, Classification and Regression Trees. Belmont, CA, USA: Wadsworth, 1984.

[12] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," ACM Queue, vol. 14, no. 1, pp. 70-93, 2016.

[13] Prometheus Authors, "Prometheus - Monitoring system & time series database," Documentation. [Online]. Available: https://prometheus.io/docs/

[14] Docker Inc., "Docker Compose overview," Docker Documentation. [Online]. Available: https://docs.docker.com/compose/

[15] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA, USA: O'Reilly Media, 2016.

[16] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.